AMT Horizon

Functional Specification / Technical Specification Document

DOCUMENT VERSION 1.3

14/01/2026

Author

Name	Role
Alexis SANTOS	Program Manager / Tech Lead

Revision History

Date	Version	Description of Changes
04/01/2026	1.0	• Document creation and drafting of sections Overview, A to E. • Added Functional Requirements.
08/01/2026	1.1	• Translated code and technical sections to English. • Adapted for NextJS, Vitest, Neon, Drizzle, better-auth.
14/01/2026	1.2	• Separated Functional and Technical Specifications into distinct documents. • Added detailed Database Structure and API Endpoints sections.
14/01/2026	1.3	• Prioritized actions over API endpoints. • Added detailed comments and explanations for tables and enums.

▶ Table of Contents

I. Database Structure (PostgreSQL/Neon)

A. Conceptual Schema

The database schema for AMT Horizon is designed to support a comprehensive taxi management system. Each table is tailored to handle specific aspects of the application, from user management to ride tracking, billing, and notifications.

Key Tables and Their Purpose

1. Users Table

```
-- Users Table: Stores all user accounts with roles, verification status,
and vehicle details for drivers.
CREATE TABLE "users" (
    "id" text PRIMARY KEY NOT NULL,
```

```
    "name" text NOT NULL,
    "email" text NOT NULL,
    "email_verified" boolean DEFAULT false NOT NULL,
    "image" text,
    "created_at" timestamp DEFAULT now() NOT NULL,
    "updated_at" timestamp DEFAULT now() NOT NULL,
    "role" "user_role",
    "banned" boolean DEFAULT false,
    "ban_reason" text,
    "ban_expires" timestamp,
    "vehicle_type" "vehicle_type",
    "vehicle_plate" text,
    "vehicle_model" text,
    "vehicle_color" text,
    CONSTRAINT "users_email_unique" UNIQUE("email")
);
```

Purpose: Stores all user accounts, including roles, verification status, and vehicle details for drivers. This table is central to user management and authentication.

2. Rides Table

```
-- Rides Table: Stores ride details, including departure, destination,
status, and associated options.
CREATE TABLE "rides" (
    "id" serial PRIMARY KEY NOT NULL,
    "departure" varchar(255) NOT NULL,
    "destination" varchar(255) NOT NULL,
    "departure_time" timestamp NOT NULL,
    "arrival_time" timestamp,
    "distance_km" numeric(10, 2),
    "price" numeric(10, 2) NOT NULL,
    "status" "ride_status" DEFAULT 'pending' NOT NULL,
    "photo_url" varchar(512),
    "created_at" timestamp DEFAULT now() NOT NULL,
    "driver_id" text,
    "customer_notes" text,
    "production" text,
    "project" text,
    "waiting_time" integer DEFAULT 0,
    "options" jsonb DEFAULT '[]'::jsonb
);
```

Purpose: Stores ride details, including departure, destination, status, and associated options. This table is key for ride management and tracking.

3. Ride Customers Table

```
-- Ride Customers Table: Links rides to customers, allowing multiple
customers per ride.
CREATE TABLE "ride_customers" (
    "id" serial PRIMARY KEY NOT NULL,
    "ride_id" integer NOT NULL,
    "customer_id" text NOT NULL,
    "created_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Links rides to customers, allowing multiple customers per ride. This supports group rides and shared billing.

4. Ride Options Table

```
-- Ride Options Table: Defines available ride options (e.g., VIP, baby
seat) and their additional prices.
CREATE TABLE "ride_options" (
    "id" serial PRIMARY KEY NOT NULL,
    "name" "option_type" NOT NULL,
    "description" text,
    "additional_price" numeric(10, 2) DEFAULT '0' NOT NULL
);
```

Purpose: Defines available ride options (e.g., VIP, baby seat) and their additional prices. This allows for customizable ride experiences.

5. Ride Selected Options Table

```
-- Ride Selected Options Table: Links rides to selected options, storing
the price at the time of selection.
CREATE TABLE "rideSelectedOptions" (
    "id" serial PRIMARY KEY NOT NULL,
    "ride_id" integer NOT NULL,
    "option_name" "option_type" NOT NULL,
    "price" numeric(10, 2) NOT NULL
);
```

Purpose: Links rides to selected options, storing the price at the time of selection. This ensures accurate billing for optional services.

6. Assignment Requests Table

```
-- Assignment Requests Table: Tracks driver requests to be assigned to
rides.
CREATE TABLE "assignment_requests" (
```

```
    "id" serial PRIMARY KEY NOT NULL,
    "ride_id" integer NOT NULL,
    "driver_id" text NOT NULL,
    "status" "request_status" DEFAULT 'pending' NOT NULL,
    "requested_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Tracks driver requests to be assigned to rides. This supports the dynamic assignment process.

7. Invoices Table

```
-- Invoices Table: Stores invoice details, including ride associations,
amounts, and payment status.
CREATE TABLE "invoices" (
    "id" serial PRIMARY KEY NOT NULL,
    "ride_id" integer NOT NULL,
    "waiting_fee" numeric(10, 2) DEFAULT '0' NOT NULL,
    "sub_total" numeric(10, 2) NOT NULL,
    "tax" numeric(10, 2) DEFAULT '0' NOT NULL,
    "total" numeric(10, 2) NOT NULL,
    "status" "invoice_status" DEFAULT 'unpaid' NOT NULL,
    "invoice_date" timestamp DEFAULT now() NOT NULL,
    "due_date" timestamp DEFAULT CURRENT_DATE + INTERVAL '30 days' NOT
NULL,
    "pdf_url" varchar(512),
    "sent_via_app" boolean DEFAULT false NOT NULL,
    "app_sent_at" timestamp,
    "reminders_sent" integer DEFAULT 0 NOT NULL,
    "last_reminder_message" text
);
```

Purpose: Stores invoice details, including ride associations, amounts, and payment status. This is critical for billing and financial tracking.

8. Notifications Table

```
-- Notifications Table: Stores notifications sent to users, with read
status.
CREATE TABLE "notifications" (
    "id" serial PRIMARY KEY NOT NULL,
    "user_id" text NOT NULL,
    "message" text NOT NULL,
    "is_read" boolean DEFAULT false NOT NULL,
    "created_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Stores notifications sent to users, with read status. This supports real-time communication and alerts.

9. Notification Preferences Table

```
-- Notification Preferences Table: Stores user preferences for
notification channels.
CREATE TABLE "notification_preferences" (
    "id" serial PRIMARY KEY NOT NULL,
    "user_id" text NOT NULL,
    "email" boolean DEFAULT true NOT NULL,
    "push" boolean DEFAULT true NOT NULL,
    "created_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Stores user preferences for notification channels. This allows users to customize how they receive alerts.

10. Activity Logs Table

```
-- Activity Logs Table: Logs user actions for auditing and compliance.
CREATE TABLE "activity_logs" (
    "id" serial PRIMARY KEY NOT NULL,
    "user_id" text NOT NULL,
    "action" varchar(255) NOT NULL,
    "details" text,
    "created_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Logs user actions for auditing and compliance. This is essential for security and GDPR compliance.

11. Shift Planning Table

```
-- Shift Planning Table: Manages driver shift schedules, linking to
productions and projects.
CREATE TABLE "shiftPlanning" (
    "id" serial PRIMARY KEY NOT NULL,
    "driver_id" text NOT NULL,
    "start_time" timestamp NOT NULL,
    "end_time" timestamp NOT NULL,
    "status" "shift_status" DEFAULT 'planned',
    "production_id" text,
    "project_id" text
);
```

Purpose: Manages driver shift schedules, linking to productions and projects. This supports workforce management and scheduling.

12. Productions Table

```
-- Productions Table: Stores production details, such as address and
contact information.
CREATE TABLE "productions" (
    "id" text PRIMARY KEY NOT NULL,
    "name" text NOT NULL,
    "address" text,
    "contact_name" text,
    "contact_email" text NOT NULL,
    "contact_phone" text,
    "created_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Stores production details, such as address and contact information. This supports project and production management.

13. Projects Table

```
-- Projects Table: Stores project details, linked to productions.
CREATE TABLE "projects" (
    "id" text PRIMARY KEY NOT NULL,
    "name" text NOT NULL,
    "production_id" text NOT NULL,
    "start_date" timestamp NOT NULL,
    "end_date" timestamp NOT NULL,
    "created_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Stores project details, linked to productions. This supports project tracking and management.

14. Account Table

```
-- Account Table: Manages user authentication accounts and tokens.
CREATE TABLE "account" (
    "id" text PRIMARY KEY NOT NULL,
    "account_id" text NOT NULL,
    "provider_id" text NOT NULL,
    "user_id" text NOT NULL,
    "access_token" text,
    "refresh_token" text,
    "id_token" text,
    "access_token_expires_at" timestamp,
    "refresh_token_expires_at" timestamp,
```

```
    "scope" text,
    "password" text,
    "created_at" timestamp DEFAULT now() NOT NULL,
    "updated_at" timestamp NOT NULL
);
```

Purpose: Manages user authentication accounts and tokens. This is essential for secure user authentication.

15. Session Table

```
-- Session Table: Manages user sessions, including tokens and IP
addresses.
CREATE TABLE "session" (
    "id" text PRIMARY KEY NOT NULL,
    "expires_at" timestamp NOT NULL,
    "token" text NOT NULL,
    "created_at" timestamp DEFAULT now() NOT NULL,
    "updated_at" timestamp NOT NULL,
    "ip_address" text,
    "user_agent" text,
    "user_id" text NOT NULL,
    "impersonated_by" text,
    CONSTRAINT "session_token_unique" UNIQUE("token")
);
```

Purpose: Manages user sessions, including tokens and IP addresses. This supports session management and security.

16. Verification Table

```
-- Verification Table: Stores verification tokens for email and other
validations.
CREATE TABLE "verification" (
    "id" text PRIMARY KEY NOT NULL,
    "identifier" text NOT NULL,
    "value" text NOT NULL,
    "expires_at" timestamp NOT NULL,
    "created_at" timestamp DEFAULT now() NOT NULL,
    "updated_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Stores verification tokens for email and other validations. This supports user verification processes.

17. Ratings Table

```
-- Ratings Table: Stores customer ratings and comments for rides.
CREATE TABLE "ratings" (
    "id" serial PRIMARY KEY NOT NULL,
    "ride_id" integer NOT NULL,
    "customer_id" text NOT NULL,
    "rating" integer NOT NULL,
    "comment" text,
    "created_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Stores customer ratings and comments for rides. This supports quality control and driver performance tracking.

B. Enums Types

Enums are used to define a set of named values. They ensure data integrity by restricting the values that can be inserted into certain columns.

Key Enums and Their Purpose

1. User Role Enum

```
-- User Role Enum: Defines the roles a user can have in the system.
CREATE TYPE "public"."user_role" AS ENUM('admin', 'driver', 'customer');--
> statement-breakpoint
```

Purpose: Defines the roles a user can have in the system, ensuring role-based access control.

2. Ride Status Enum

```
-- Ride Status Enum: Defines the possible statuses of a ride.
CREATE TYPE "public"."ride_status" AS ENUM('pending', 'assigned',
'completed', 'cancelled');--> statement-breakpoint
```

Purpose: Defines the possible statuses of a ride, supporting ride lifecycle management.

3. Request Status Enum

```
-- Request Status Enum: Defines the possible statuses of an assignment
request.
CREATE TYPE "public"."request_status" AS ENUM('pending', 'approved',
'rejected');--> statement-breakpoint
```

Purpose: Defines the possible statuses of an assignment request, supporting the assignment workflow.

4. Invoice Status Enum

```
-- Invoice Status Enum: Defines the possible statuses of an invoice.
CREATE TYPE "public"."invoice_status" AS ENUM('unpaid', 'paid',
'cancelled');--> statement-breakpoint
```

Purpose: Defines the possible statuses of an invoice, supporting billing and payment tracking.

5. Shift Status Enum

```
-- Shift Status Enum: Defines the possible statuses of a driver's shift.
CREATE TYPE "public"."shift_status" AS ENUM('planned', 'active',
'completed', 'cancelled');--> statement-breakpoint
```

Purpose: Defines the possible statuses of a driver's shift, supporting shift management.

6. Option Type Enum

```
-- Option Type Enum: Defines the possible options that can be selected for
a ride.
CREATE TYPE "public"."option_type" AS ENUM('vip', 'baby_seat', 'van',
'luxury', 'pet_friendly', 'extra_luggage', 'wheelchair_accessible',
'motorbike');--> statement-breakpoint
```

Purpose: Defines the possible options that can be selected for a ride, supporting ride customization.

7. Vehicle Type Enum

```
-- Vehicle Type Enum: Defines the possible types of vehicles a driver can
have.
CREATE TYPE "public"."vehicle_type" AS ENUM('sedan', 'suv', 'van',
'motorbike', 'luxury');--> statement-breakpoint
```

Purpose: Defines the possible types of vehicles a driver can have, supporting vehicle management.

C. Explanations

Tables

Each table in the database is designed to handle a specific aspect of the application, ensuring that data is organized, accessible, and secure. The relationships between tables allow for complex queries and

operations, supporting the full range of functionalities required by AMT Horizon.

Enums

Enums provide a way to enforce data consistency and integrity by restricting the values that can be stored in certain columns. This ensures that only valid and expected values are used throughout the application.

II. SQL Query Examples

Here are some practical SQL query examples that demonstrate how to interact with the database:

A. List unassigned rides

```sql
SELECT * FROM rides
WHERE status = 'pending' AND driver_id IS NULL;
```

Purpose: Retrieve all rides that have not yet been assigned to a driver.

B. Generate an invoice

```sql
INSERT INTO invoices (ride_id, amount, status, invoice_date, due_date)
VALUES (1, 25.50, 'unpaid', NOW(), NOW() + INTERVAL '30 days');
```

Purpose: Create a new invoice for a ride, setting the due date to 30 days from now.

C. Update assignment request status

```sql
UPDATE assignment_requests
SET status = 'approved'
WHERE request_id = 1;
```

Purpose: Approve a driver's request to be assigned to a ride.

D. Retrieve user's unread notifications

```sql
SELECT * FROM notifications
WHERE user_id = 1 AND is_read = FALSE;
```

Purpose: Fetch all unread notifications for a specific user.

E. List available drivers within a 5 km radius

```
-- Assuming `latitude` and `longitude` columns in the users table
SELECT u.user_id, u.email
FROM users u
WHERE u.role = 'driver'
  AND u.available = TRUE
  AND ST_DWithin(
      ST_MakePoint(u.longitude, u.latitude)::geography,
      ST_MakePoint(:client_longitude, :client_latitude)::geography,
      5000  -- 5 km in meters
  );
```

Purpose: Find all available drivers within a 5 km radius of a specified location.

F. Calculate driver's average rating

```
SELECT AVG(rating) as average_rating
FROM ratings
WHERE driver_id = :driver_id;
```

Purpose: Calculate the average rating for a specific driver based on customer feedback.

G. Send invoice via application

```
UPDATE invoices
SET
    sent_via_app = TRUE,
    app_sent_at = NOW(),
    status = 'sent'
WHERE invoice_id = :invoice_id;
-- Send PDF by email and in-app notification (backend logic)
```

Purpose: Mark an invoice as sent via the application and update its status.

H. List invoices eligible for reminder (unpaid, overdue)

```
SELECT invoice_id, ride_id, amount, due_date
FROM invoices
WHERE status = 'unpaid'
  AND due_date < NOW()
  AND reminders_sent < 3;
```

Purpose: Identify all unpaid and overdue invoices that are eligible for a reminder.

I. Send manual reminder

```
UPDATE invoices
SET
    reminders_sent = reminders_sent + 1,
    last_reminder_message = :custom_message,
    status = 'reminder_sent'
WHERE invoice_id = :invoice_id;
-- Send email/notification with custom message
```

Purpose: Send a manual reminder for an overdue invoice and update its status.

III. Glossary

- **2FA**: Two-Factor Authentication, security method requiring two proofs of identity.
- **Activity Logs**: Records of user actions within the system, used for auditing and compliance with regulations like GDPR.
- **Assignment Request**: A request made by a driver to be assigned to a specific ride, tracked in the `assignment_requests` table.
- **Database Schema**: The structure of the database, including tables, columns, and relationships, designed to support the AMT Horizon application.
- **Enum (Enumeration)**: A data type consisting of a set of named values, used to ensure data integrity by restricting column values to a predefined list.
- **GDPR**: General Data Protection Regulation, legal framework for personal data protection in Europe.
- **Geospatial Query**: A query that involves geographic data, such as finding drivers within a specific radius using `ST_DWithin`.
- **Invoice**: A document detailing the cost of a ride, including taxes, fees, and payment status, stored in the `invoices` table.
- **JWT (JSON Web Token)**: A compact, URL-safe means of representing claims to be transferred between two parties, often used for authentication.
- **Mapbox/Google Maps**: Mapping APIs for geolocation and route display.
- **Neon**: A serverless PostgreSQL service optimized for scalable and modern applications.
- **Notification**: An alert sent to users regarding events like ride assignments, invoices, or urgent updates, stored in the `notifications` table.
- **OCR (Optical Character Recognition)**: Technology used to extract text from images, such as tickets or invoices, for ride data import.
- **PostgreSQL**: An open-source relational database management system used to store and manage the data for AMT Horizon.
- **PostGIS**: A spatial database extender for PostgreSQL, adding support for geographic objects and geospatial queries.
- **Primary Key**: A column or set of columns that uniquely identifies each row in a table.
- **Query**: A request for data or an action on the data stored in the database, such as retrieving unassigned rides or updating an invoice status.

- **Ratings**: Customer feedback on rides, stored in the `ratings` table, used for quality control and driver performance tracking.
- **REST API**: Representational State Transfer Application Programming Interface, an architectural style for designing networked applications.
- **Ride**: A taxi trip with details like departure, destination, time, price, and status, stored in the `rides` table.
- **Ride Options**: Additional services (e.g., VIP, baby seat) that can be selected for a ride, stored in the `ride_options` and `rideSelectedOptions` tables.
- **Role-Based Access Control**: A method of restricting access to features based on user roles (Admin, Driver, Customer).
- **Schema**: The organization of data as a blueprint of how the database is constructed, including tables and relationships.
- **Session**: A period during which a user is logged into the application, managed in the `session` table.
- **Shift Planning**: Management of driver shift schedules, including start/end times and status, stored in the `shiftPlanning` table.
- **SQL (Structured Query Language)**: A standard language for managing and manipulating relational databases.
- **Stripe/PayPal**: Secure online payment solutions.
- **ST_DWithin**: A PostgreSQL/PostGIS function used for geospatial queries, such as finding drivers within a specific radius.
- **Table**: A structured set of data organized into rows and columns, such as the `users` or `rides` table.
- **User Role**: Classification of users in the system (Admin, Driver, Customer), determining access and permissions.
- **Verification**: Process of validating user information, such as email, stored in the `verification` table.